

PROJECT DOCUMENTATION

Software Development Life Cycle

Gaming Lounge Manager — "Play N Slay"

VERSION

1.0.0

DATE

September 2026

SDLC MODEL

Waterfall

LICENSE

MIT

Table of Contents

01 Planning

02 Requirements Analysis

03 System Design

04 Implementation (Coding)

05 Testing

06 Deployment

07 Maintenance

Phase 1 — Planning

1.1 Project Overview

ATTRIBUTE	DETAIL
Project Name	Gaming Lounge Manager (Play N Slay)
Project Type	Full-Stack Web Application
Domain	Gaming Lounge / Café Management
License	MIT
Version	1.0.0

1.2 Problem Statement

Gaming lounges and cafés require a centralized system to manage station bookings, track inventory (snacks, drinks), handle multi-role access (admin, employee, member), and generate real-time revenue analytics. Manual management leads to double-bookings, inventory discrepancies, and lost revenue visibility.

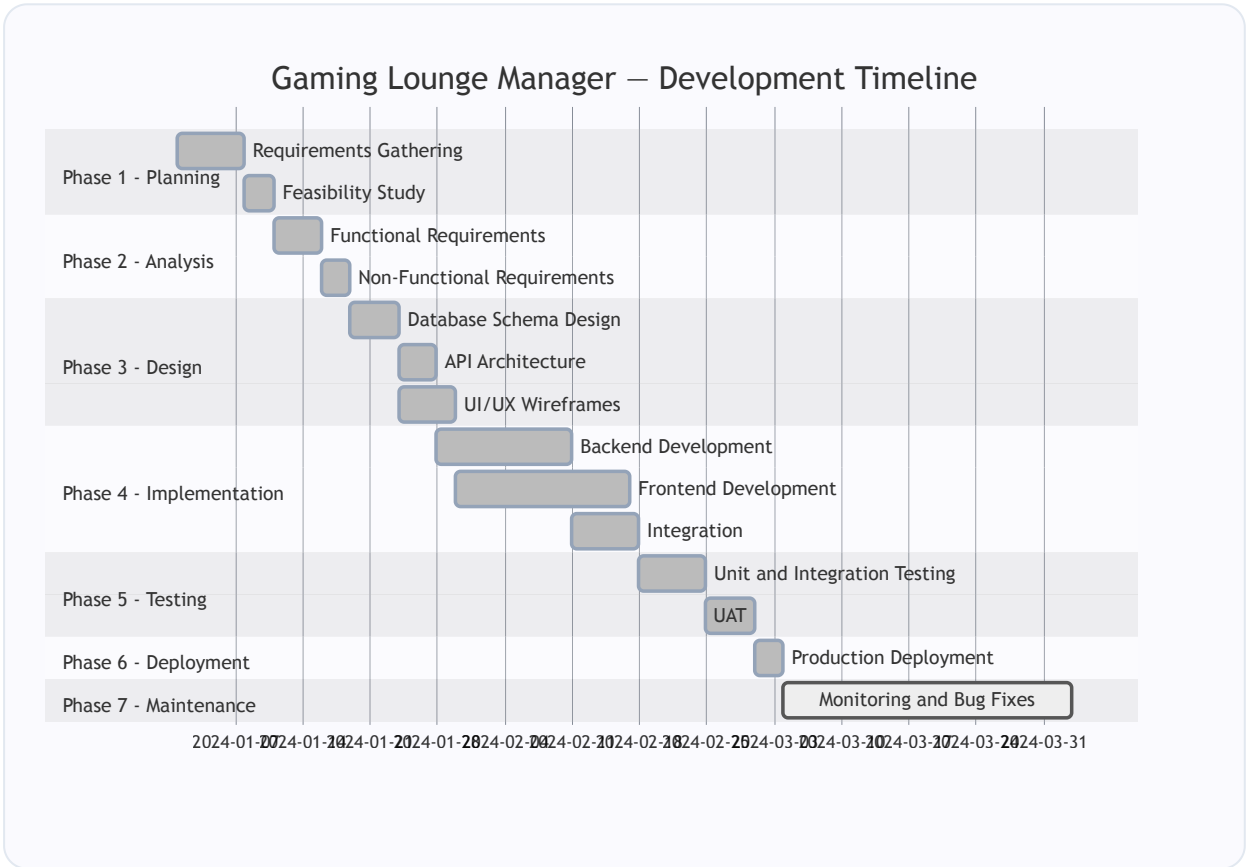
1.3 Project Objectives

- **Automate station booking** with time-slot conflict detection across 8+ gaming station types (PC, PS4, PS5, XBOX, VR, Racing Sim, etc.)
- **Role-based access control** for three distinct user types: Admin, Employee, and Member
- **Real-time inventory tracking** with low-stock alerts and per-booking item attachment
- **Comprehensive analytics dashboard** with revenue splits (online vs. offline), employee performance metrics, and inventory valuation
- **Dual booking channels** — online self-service for members and offline walk-in entry for employees
- **Dynamic pricing engine** supporting flat-rate and per-player pricing models with tiered discounts

1.4 Feasibility Study

FEASIBILITY TYPE	ASSESSMENT
Technical	✅ Feasible — Uses proven, modern technologies (React, Express, PostgreSQL, TypeScript)
Operational	✅ Feasible — Three clear user roles with well-defined permissions
Economic	✅ Feasible — Open-source stack, single PostgreSQL instance, Cloudinary free tier for images
Schedule	✅ Feasible — Modular architecture allows incremental delivery

1.5 Project Timeline



1.6 Stakeholders

STAKEHOLDER	ROLE	INTEREST
Lounge Owner	Project Sponsor	Revenue tracking, business insights
Admin Users	System Administrator	Full management of stations, inventory, users, bookings
Employees	Operational Staff	Walk-in booking entry, session timers, daily stats
Members (Customers)	End Users	Online booking, viewing history, profile management

Phase 2 — Requirements Analysis

2.1 Functional Requirements

FR-01: User Authentication & Authorization

ID	REQUIREMENT	PRIORITY
FR-01.1	Users can register with username and password	HIGH
FR-01.2	Users can log in with credentials (username or email)	HIGH
FR-01.3	Role-based access: <code>admin</code> , <code>employee</code> , <code>member</code>	HIGH
FR-01.4	Admin can reset any user's password	MEDIUM
FR-01.5	Users can change their own password	MEDIUM
FR-01.6	Rate limiting on login (10 attempts/15 min) and registration (5/hour)	HIGH
FR-01.7	Session-based authentication with 30-day cookie expiry	HIGH

FR-02: Gaming Station Management

ID	REQUIREMENT	PRIORITY
FR-02.1	Admin can create, update, delete gaming station types	HIGH
FR-02.2	Each station has: name, description, hourly price, max players, price model, image	HIGH
FR-02.3	Station types support <code>flat</code> and <code>per_player</code> pricing models	HIGH
FR-02.4	Admin can manage a games catalog (title, platforms, genre, player range, image)	MEDIUM
FR-02.5	Stations and games can be marked active/inactive	MEDIUM

FR-03: Booking System

ID	REQUIREMENT	PRIORITY
FR-03.1	Members can create bookings by selecting station type, time slot, player count	HIGH
FR-03.2	Time-slot conflict detection prevents double-booking on the same station	HIGH
FR-03.3	Auto-generated unique booking reference (e.g., <code>A1B2C3D4</code>)	HIGH
FR-03.4	Booking status workflow: <code>Pending</code> → <code>Approved</code> → <code>Completed</code> or <code>Cancelled</code>	HIGH
FR-03.5	Employees can create offline walk-in bookings (auto-creates customer if new)	HIGH
FR-03.6	Offline bookings are prefixed with <code>OFF-</code> for identification	MEDIUM
FR-03.7	Staff can start/stop session timers on bookings	MEDIUM
FR-03.8	Inventory items (snacks, drinks) can be attached to bookings	MEDIUM
FR-03.9	Booking price auto-calculated using the dynamic pricing engine	HIGH

FR-04: Inventory Management

ID	REQUIREMENT	PRIORITY
FR-04.1	Staff can add, update, delete inventory items	HIGH
FR-04.2	Each item has: name, category, buying price, selling price, stock quantity	HIGH
FR-04.3	Low-stock threshold alerts (configurable per item, default = 5)	MEDIUM
FR-04.4	Stock auto-deducted when items are attached to a booking	HIGH

FR-05: Analytics & Reporting

ID	REQUIREMENT	PRIORITY
FR-05.1	Admin dashboard shows total revenue, total bookings, low-stock count	HIGH
FR-05.2	Revenue split between online and offline bookings	MEDIUM
FR-05.3	Employee performance metrics (bookings count, revenue generated)	MEDIUM
FR-05.4	Inventory valuation report (stock × buying price)	MEDIUM
FR-05.5	Export bookings to CSV	MEDIUM
FR-05.6	Employee personal stats: daily entries, total entries, total revenue	MEDIUM

FR-06: User Profile Management

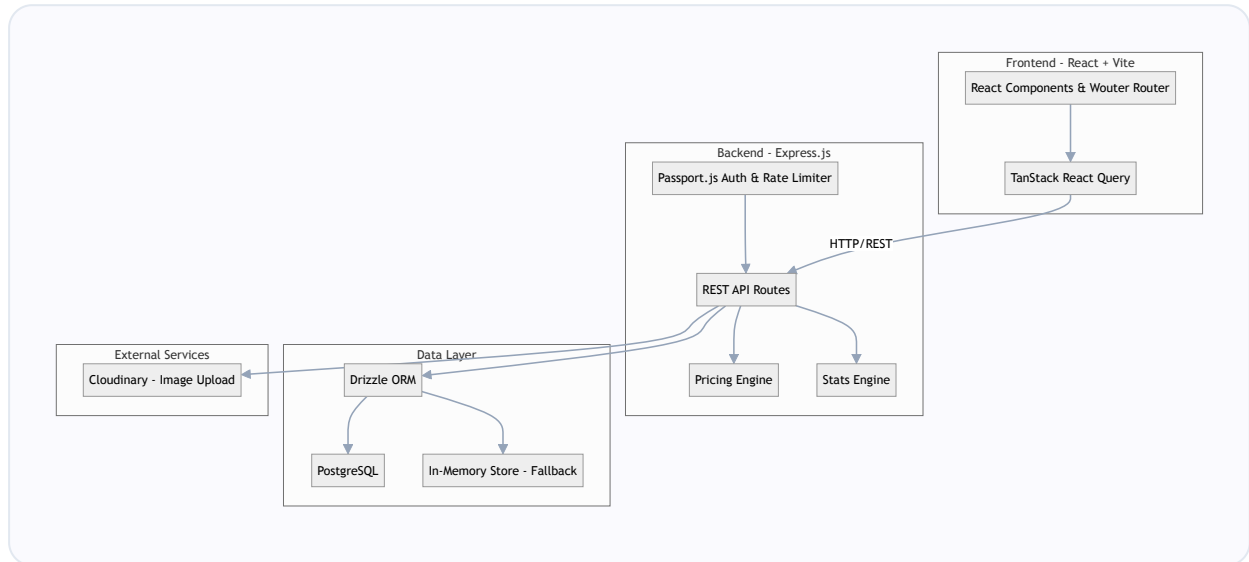
ID	REQUIREMENT	PRIORITY
FR-06.1	Users can update full name, phone, avatar	MEDIUM
FR-06.2	Image uploads via Cloudinary	MEDIUM
FR-06.3	Membership tiers: Bronze, Silver, Gold, Platinum	LOW

2.2 Non-Functional Requirements

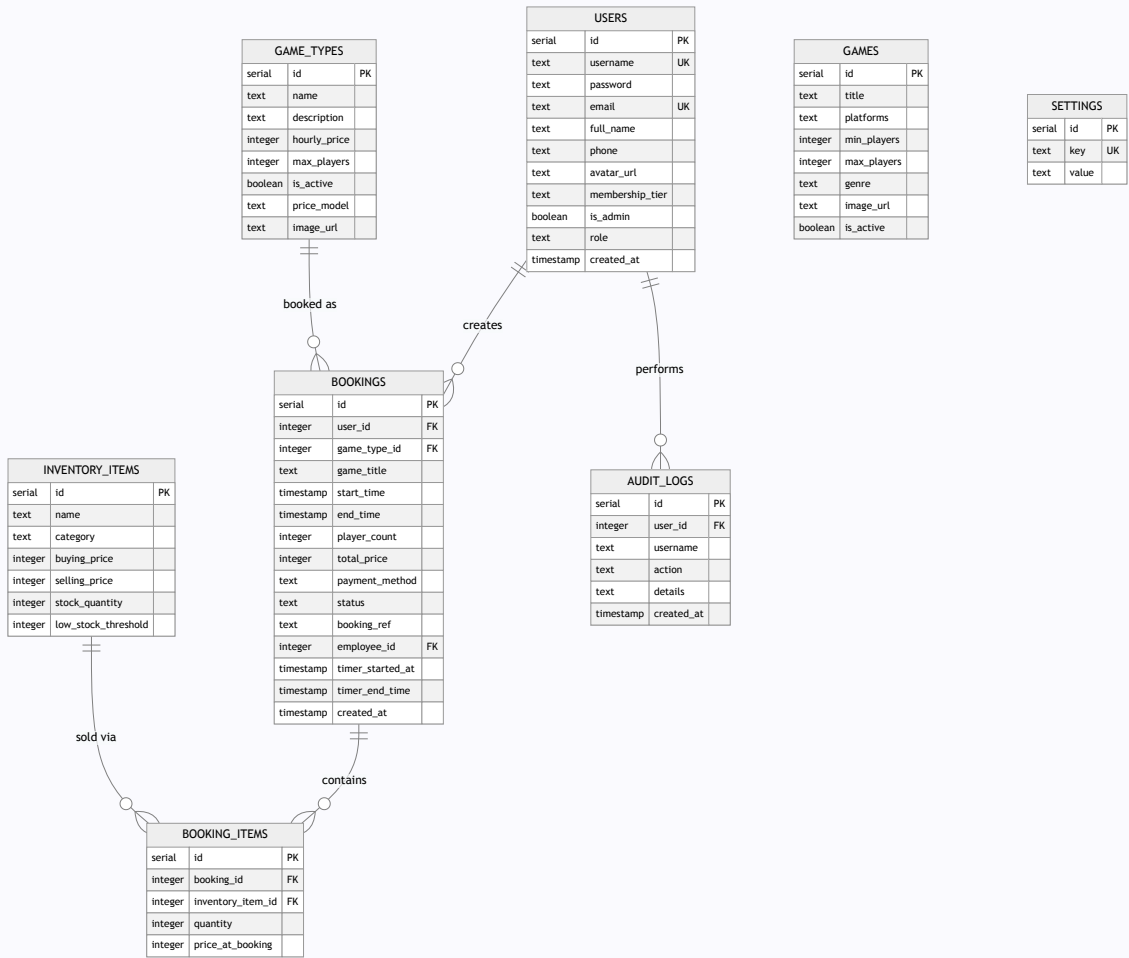
ID	REQUIREMENT	CATEGORY	DETAIL
NFR-01	Performance	Response Time	API responses < 500ms under normal load
NFR-02	Security	Password Hashing	bcrypt with 64-byte key + random salt (timing-safe comparison)
NFR-03	Security	Rate Limiting	Express rate-limit on auth endpoints
NFR-04	Security	Session	Server-side sessions stored in PostgreSQL
NFR-05	Scalability	Database	PostgreSQL with Drizzle ORM (supports migration via drizzle-kit push)
NFR-06	Usability	UI	Gaming-themed dark mode with Orbitron/Rajdhani fonts, Framer Motion animations
NFR-07	Portability	Storage	Dual storage backends (PostgreSQL production, in-memory fallback)
NFR-08	Maintainability	Type Safety	End-to-end TypeScript with Zod schema validation

Phase 3 — System Design

3.1 Architecture Overview



3.2 Database Schema (ER Diagram)

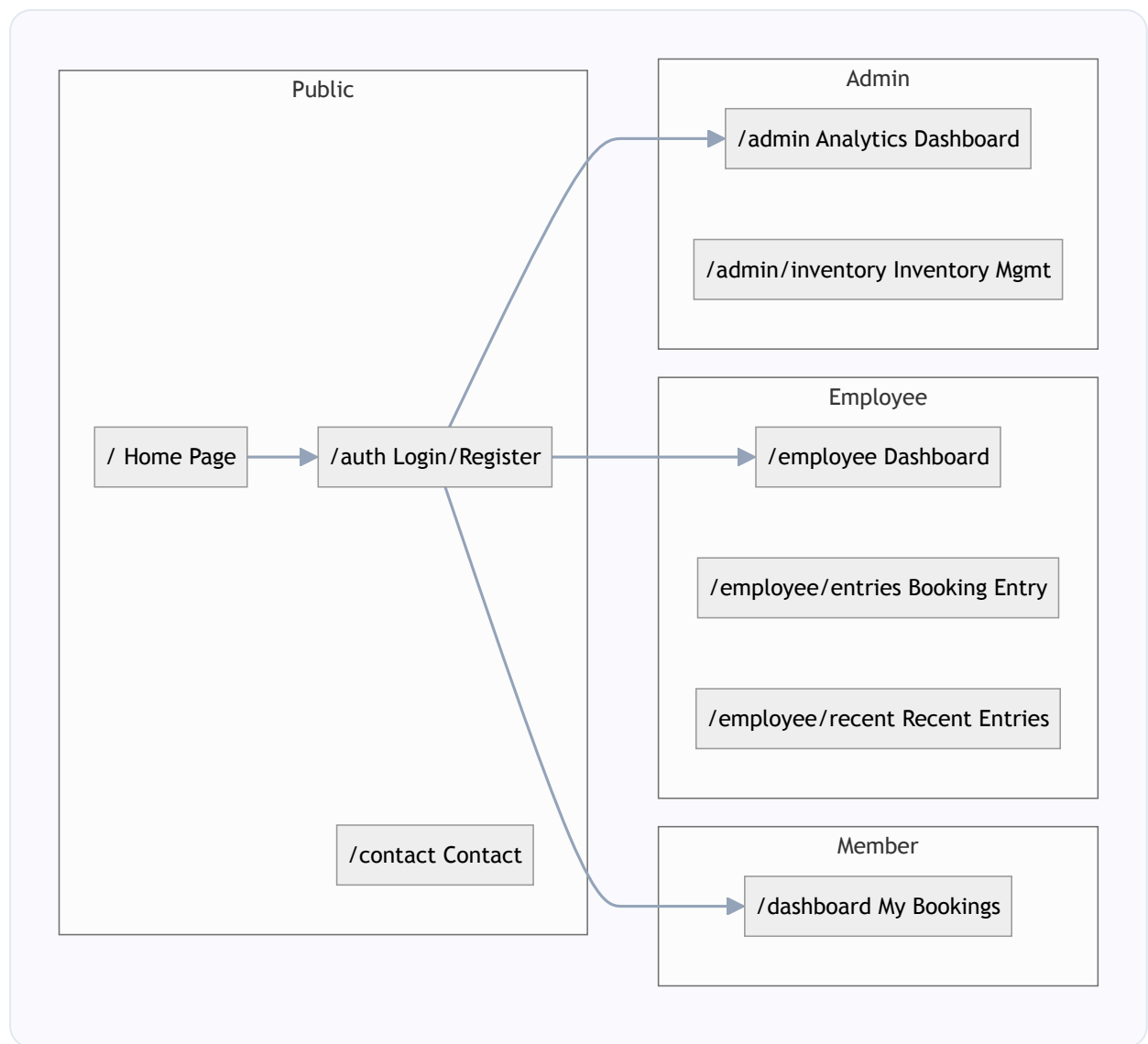


3.3 API Design

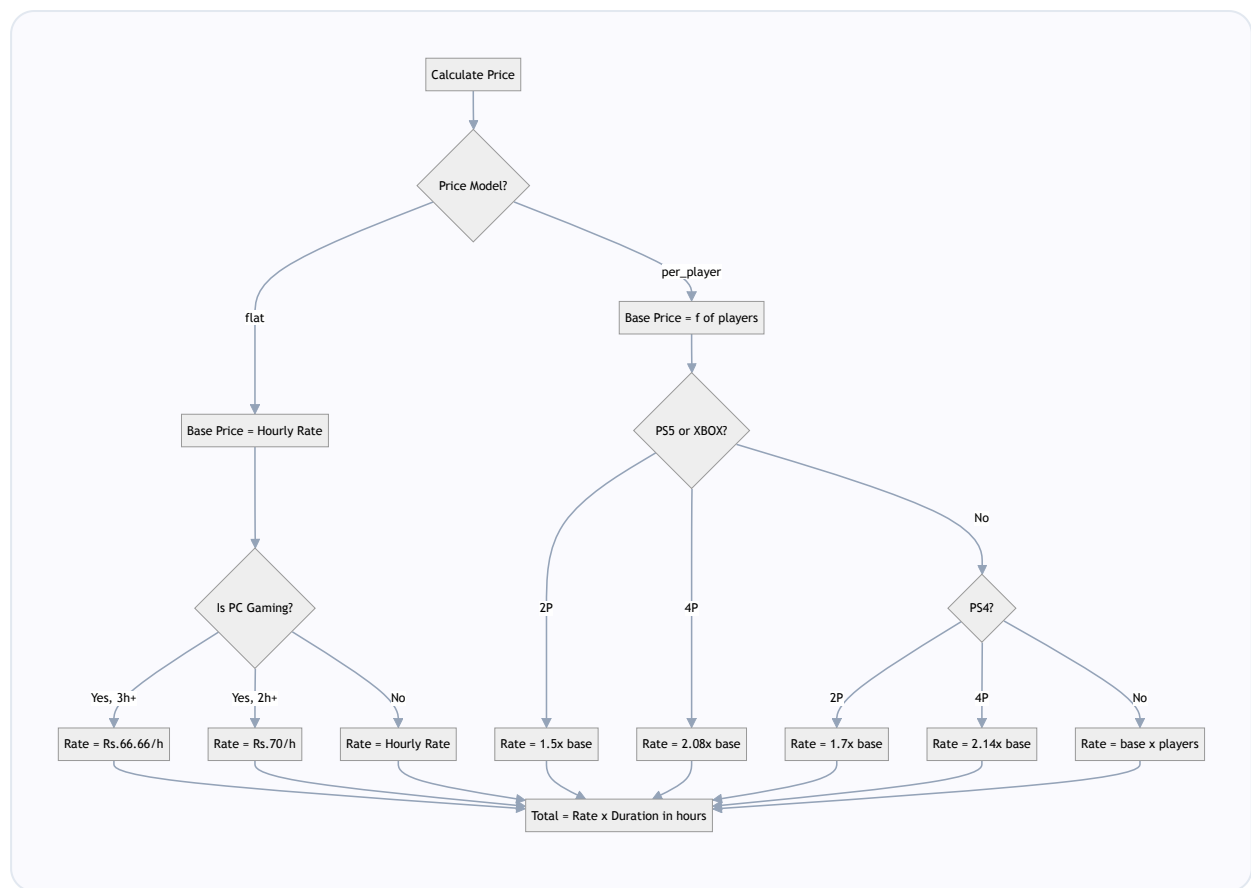
METHOD	ENDPOINT	AUTH	ROLE	DESCRIPTION
POST	/api/register	✗	Public	Create new member account
POST	/api/login	✗	Public	Authenticate user
POST	/api/logout	✓	Any	End session
GET	/api/user	✓	Any	Get current user
PATCH	/api/user/profile	✓	Any	Update profile
POST	/api/user/change-password	✓	Any	Change own password
GET	/api/game-types	✗	Public	List all station types
POST	/api/game-types	✓	Admin	Create station type
PATCH	/api/game-types/:id	✓	Admin	Update station type
DELETE	/api/game-types/:id	✓	Admin	Delete station type
GET	/api/games	✗	Public	List all games
POST	/api/games	✓	Admin	Add game to catalog
PATCH	/api/games/:id	✓	Admin	Update game
DELETE	/api/games/:id	✓	Admin	Delete game
GET	/api/bookings	✓	Any	List bookings (scoped by role)
POST	/api/bookings	✓	Any	Create online booking
PATCH	/api/bookings/:id/status	✓	Staff	Update booking status

METHOD	ENDPOINT	AUTH	ROLE	DESCRIPTION
POST	/api/bookings/:id/timer/start	✓	Staff	Start session timer
POST	/api/bookings/:id/timer/stop	✓	Staff	Stop session timer
GET	/api/bookings/:id/items	✓	Any	Get booking's attached items
POST	/api/bookings/offline	✓	Staff	Create walk-in booking
GET	/api/inventory	✗	Public	List inventory items
POST	/api/inventory	✓	Staff	Add inventory item
PATCH	/api/inventory/:id	✓	Admin	Update inventory item
DELETE	/api/inventory/:id	✓	Admin	Delete inventory item
GET	/api/admin/stats/comprehensive	✓	Admin	Full analytics dashboard
GET	/api/employee/stats	✓	Employee	Personal performance stats
GET	/api/admin/users	✓	Admin	List all users
POST	/api/admin/users/:id/reset-password	✓	Admin	Reset user password
GET	/api/admin/bookings/export	✓	Admin	Export bookings as CSV
POST	/api/upload	✓	Staff	Upload image to Cloudinary

3.4 Frontend Page Architecture



3.5 Dynamic Pricing Model



3.6 Technology Stack

LAYER	TECHNOLOGY	PURPOSE
Language	TypeScript 5.6	End-to-end type safety
Frontend Framework	React 18	Component-based UI
Routing	Wouter	Lightweight client-side routing
State/Data	TanStack React Query	Server state management & caching
UI Components	Radix UI + shadcn/ui	Accessible, headless component primitives
Styling	Tailwind CSS 3.4	Utility-first CSS framework
Animations	Framer Motion	Page transitions and micro-interactions
Charts	Recharts	Analytics visualizations
Forms	React Hook Form + Zod	Form management with schema validation
Backend Framework	Express.js 4	REST API server
Authentication	Passport.js (Local Strategy)	Session-based auth
ORM	Drizzle ORM	Type-safe database queries
Database	PostgreSQL	Persistent relational data storage
Schema Validation	Zod + drizzle-zod	Runtime validation from DB schema
Image Storage	Cloudinary	CDN-backed image hosting
File Upload	Multer	Multipart form data handling
Build Tool	Vite 7 + esbuild	Frontend bundling + server compilation
Fonts	Orbitron + Rajdhani	Gaming-themed typography

Phase 4 — Implementation (Coding)

4.1 Project Structure

```
Gaming-Lounge-Manager/
├── client/                                # Frontend application
│   ├── src/
│   │   ├── App.tsx                      # Root component, routing
│   │   ├── main.tsx                     # Entry point
│   │   ├── index.css                    # Global styles
│   │   └── pages/
│   │       ├── home-page.tsx            # Landing page
│   │       ├── auth-page.tsx            # Login / Register
│   │       ├── dashboard-page.tsx        # Member bookings
│   │       ├── admin-dashboard.tsx       # Admin analytics
│   │       ├── employee-dashboard.tsx    # Employee portal
│   │       ├── booking-entry-page.tsx    # Walk-in booking form
│   │       ├── inventory-page.tsx        # Inventory management
│   │       ├── contact-page.tsx         # Contact form
│   │       └── not-found.tsx            # 404 page
│   │   ├── components/
│   │       ├── booking-wizard.tsx        # Multi-step booking form
│   │       ├── game-card.tsx            # Station display card
│   │       ├── invoice-modal.tsx         # Booking invoice
│   │       ├── profile-modal.tsx         # User profile editor
│   │       ├── image-upload.tsx         # Cloudinary uploader
│   │       ├── layout-navbar.tsx         # Navigation bar
│   │       ├── employee-sidebar.tsx      # Employee side nav
│   │       └── ui/                       # shadcn/ui components
│   │   ├── hooks/
│   │   └── lib/
│   └── index.html
├── server/                                # Backend application
│   ├── index.ts                         # Server entry point
│   ├── routes.ts                        # API route handlers + seed data
│   ├── auth.ts                          # Authentication setup
│   ├── storage.ts                       # Database + memory storage
│   ├── db.ts                           # PostgreSQL connection
│   ├── stats.ts                         # Analytics computation
│   ├── cloudinary.ts                    # Image upload service
│   └── seed.ts                          # Initial data seeding
```

```

|   ├── static.ts           # Static file serving
|   └── vite.ts             # Vite dev middleware
├── shared/                 # Shared between client & server
|   ├── schema.ts          # Drizzle DB schema + Zod validators
|   ├── routes.ts          # API path constants
|   └── pricing.ts          # Dynamic pricing engine
├── drizzle.config.ts       # ORM configuration
├── tailwind.config.ts      # Theme customization
├── vite.config.ts          # Build configuration
├── tsconfig.json           # TypeScript configuration
└── package.json            # Dependencies & scripts

```

4.2 Key Implementation Highlights

4.2.1 Dual Storage Pattern (Strategy Pattern)

The system implements the **Strategy Pattern** for data storage — a `DatabaseStorage` class (PostgreSQL via Drizzle ORM) and a `MemStorage` class (in-memory Maps), both conforming to the `IStorage` interface. The active implementation is selected at runtime:

```

// Automatic storage selection
export const storage = process.env.DATABASE_URL
  ? new DatabaseStorage()
  : new MemStorage();

```

4.2.2 Password Security

- **Hashing:** script with 64-byte derived key + 16-byte random salt
- **Comparison:** Timing-safe comparison via `crypto.timingSafeEqual()` to prevent timing attacks
- **Sanitization:** Passwords are stripped from all API responses via `sanitizeUser()`

4.2.3 Booking Conflict Detection

Before creating any booking (online or offline), the system queries existing bookings on the same station for overlapping time ranges, excluding cancelled bookings. Uses Drizzle ORM's `lt()` and `gt()` operators for range overlap detection.

4.2.4 Dynamic Pricing Engine

The shared `pricing.ts` module implements tiered, non-linear pricing:

- **PC Gaming:** Discount tiers at 2h (₹70/h) and 3h (₹66.66/h)
- **PS5/XBOX:** Multiplayer rates — 2P = 1.5×, 4P = 2.08× base

- **PS4:** Multiplayer rates — 2P = 1.7×, 4P = 2.14× base
- **Others:** Linear per-player or flat rate

4.2.5 Role-Based Middleware

Two Express middleware guards control API access:

- `requireAdmin` — Restricts to `admin` role or `isAdmin: true`
- `requireStaff` — Allows both `admin` and `employee` roles

4.3 Coding Standards

STANDARD	IMPLEMENTATION
Type Safety	End-to-end TypeScript, Zod runtime validation
Code Organization	Monorepo with shared types between client/server
State Management	Server state via React Query, no client-side Redux
Validation	Zod schemas derived from Drizzle table definitions
Error Handling	Try-catch with structured JSON error responses
Money Handling	All prices stored in cents (integer) to avoid floating-point issues
Path Constants	API paths defined once in <code>shared/routes.ts</code>

Phase 5 — Testing

5.1 Testing Strategy

TESTING TYPE	SCOPE	TOOLS/APPROACH
Type Checking	Full codebase	<code>tsc</code> (TypeScript compiler) via <code>npm run check</code>
Schema Validation	All API inputs	Zod schemas auto-derived from DB schema via drizzle-zod
Manual Functional Testing	All user flows	Browser-based testing across all 3 roles
Conflict Testing	Booking system	Verify double-booking prevention on same station
Auth Testing	Login/Register	Rate limiter verification, password hashing, session handling
Pricing Testing	All station types	Verify tiered pricing across player counts and durations

5.2 Test Scenarios

Authentication Tests

#	TEST CASE	EXPECTED RESULT
TC-01	Register with valid username/password	Account created, auto-logged in
TC-02	Register with duplicate username	400 error: "Username already exists"
TC-03	Login with correct credentials	200 + user object (no password)
TC-04	Login with wrong password	401 Unauthorized
TC-05	Exceed login rate limit (>10 in 15 min)	429: "Too many login attempts"
TC-06	Access admin route as member	403: "Admin access required"
TC-07	Change password with wrong current password	400: "Current password is incorrect"

Booking Tests

#	TEST CASE	EXPECTED RESULT
TC-08	Book PS5 for 2 players, 1 hour	Price = ₹180.00 (1.5× of ₹120)
TC-09	Book same station at overlapping time	409 Conflict with time slot details
TC-10	Employee creates offline walk-in	Booking created with OFF- prefix, auto-approved
TC-11	Offline booking for unknown customer	New member auto-created
TC-12	Book with end time before start time	400: "Invalid duration"
TC-13	Attach inventory items to booking	Stock deducted, total price includes item costs

Inventory Tests

#	TEST CASE	EXPECTED RESULT
TC-14	Add item with buying < selling price	Item created successfully
TC-15	Stock falls below low-stock threshold	Appears in low-stock count in admin stats
TC-16	Delete inventory item	Item removed, 204 response

5.3 Build Verification

```
# Type checking
npm run check

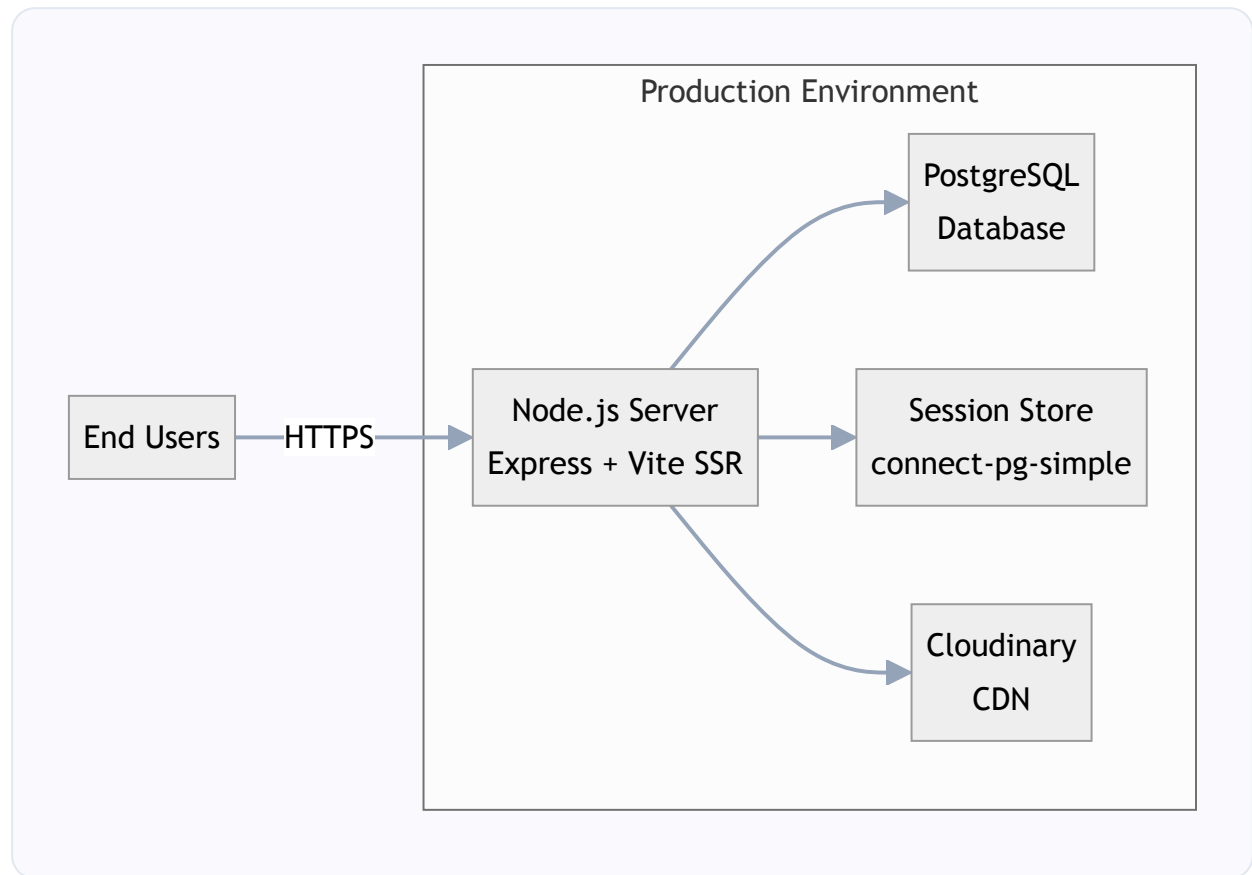
# Development server
npm run dev

# Production build
npm run build

# Database schema push
npm run db:push
```

Phase 6 — Deployment

6.1 Deployment Architecture



6.2 Environment Variables

VARIABLE	REQUIRED	DESCRIPTION
<code>DATABASE_URL</code>	Yes	PostgreSQL connection string
<code>SESSION_SECRET</code>	Yes	Express session encryption key
<code>CLOUDINARY_CLOUD_NAME</code>	Optional	Cloudinary account name
<code>CLOUDINARY_API_KEY</code>	Optional	Cloudinary API key
<code>CLOUDINARY_API_SECRET</code>	Optional	Cloudinary API secret
<code>ADMIN_EMAIL</code>	Optional	Default admin login email
<code>ADMIN_PASSWORD</code>	Optional	Default admin password
<code>EMPLOYEE_EMAIL</code>	Optional	Default employee login email
<code>EMPLOYEE_PASSWORD</code>	Optional	Default employee password
<code>NODE_ENV</code>	Optional	<code>production</code> for secure cookies

6.3 Build & Deploy Steps

```
# 1. Install dependencies
npm install

# 2. Push database schema
npm run db:push

# 3. Build for production (Vite frontend + esbuild backend)
npm run build

# 4. Start production server
NODE_ENV=production npm start
```

6.4 Auto-Seeding

On first launch, the application automatically:

1. Creates admin and employee accounts using environment variable credentials
2. Seeds 8 default gaming station types (PC, PS4, PS5, XBOX, Racing Sim, VR, IB Cricket, Meta Shot)
3. Seeds 4 default inventory items (Mountain Dew, Red Bull, Lays, KitKat)

4. Creates a mock booking for admin visibility verification

Phase 7 — Maintenance

7.1 Maintenance Activities

ACTIVITY	FREQUENCY	DESCRIPTION
Database Backups	Daily	Automated PostgreSQL backups
Dependency Updates	Monthly	Review and update npm packages for security patches
Log Monitoring	Continuous	Monitor server logs for errors and rate-limit triggers
Low-Stock Alerts	Real-time	Admin dashboard highlights items below threshold
Session Cleanup	Automatic	connect-pg-simple handles expired session cleanup
Schema Migrations	As needed	drizzle-kit push for zero-downtime schema changes

7.2 Known Limitations & Future Enhancements

AREA	CURRENT STATE	FUTURE ENHANCEMENT
Payment	Offline only, manual tracking	Integrate Razorpay/Stripe for online payments
Notifications	None	Email/SMS confirmations for bookings
Membership Tiers	Schema exists (bronze/silver/gold/platinum)	Implement tier-based discounts and loyalty points
Audit Logging	Schema exists (audit_logs table)	Implement audit trail for all admin actions
Multi-Lounge	Single location	Support multiple branches with location-based data
Mobile App	Responsive web only	React Native companion app
Real-time Updates	REST polling	WebSocket integration (ws package already installed)
Reporting	CSV export only	PDF invoice generation, scheduled email reports
Search	Basic listing	Full-text search on games catalog

7.3 Security Maintenance Checklist

- ☐ Rotate `SESSION_SECRET` periodically
- ☐ Review and update rate-limit thresholds based on traffic patterns
- ☐ Audit admin/employee account access logs
- ☐ Ensure `DATABASE_URL` uses SSL in production
- ☐ Verify Cloudinary API keys are not exposed in client bundle
- ☐ Keep Node.js runtime updated for security patches
- ☐ Change default seed passwords before production deployment

Document Version: 1.0

Project: Gaming Lounge Manager (Play N Slay) v1.0.0

SDLC Model: Waterfall (with iterative elements)

Date: September 2026